

Project Spec: In-Memory Key-Value Store

A staged C++ systems programming project — networking, concurrency, and persistence

Overview

Build a small server, similar in spirit to Redis, that listens on a TCP port and stores data in memory as key/value pairs. Clients connect over the network and issue simple text commands such as **SET**, **GET**, and **DEL**. The project is designed to be built in stages, with each stage producing a working milestone before moving on to the next.

This project touches four core systems-programming areas: data structures, networking, concurrency, and (optionally) persistence — making it a natural next step after building a standalone thread pool.

Stage 1 — Single-Threaded Core

Get the data structure and command parsing right before introducing any networking code.

- Wrap `std::unordered_map<std::string, std::string>` as the storage backend
- Build a small REPL / CLI to test **SET**, **GET**, and **DEL** locally
- Define a simple command parser that tokenizes a line of input into a command and arguments

Stage 2 — Networking

Turn the local store into a server that clients can connect to over TCP.

- Use raw POSIX sockets (`socket`, `bind`, `listen`, `accept`) — or `Boost.Asio` for less boilerplate if you'd rather focus on the store logic itself
- Parse incoming text commands over the connection and dispatch them to the store
- Write responses back to the client
- Test manually with `nc localhost 6379`

Stage 3 — Concurrency

This is the core of the project. Since you've already built a thread pool, reuse it here rather than spawning a raw thread per connection.

Locking strategies worth trying, roughly in order of sophistication:

- **Single global mutex** — simplest option, get correctness first
- **Sharded locks** — split the keyspace into N buckets, each with its own mutex, for much better throughput under contention
- **Reader-writer lock** (`std::shared_mutex`) — useful since reads will likely dominate writes

This is where the project earns its keep: sharded locking and lock contention are easy to read about, but only really click once you've profiled a bottleneck in your own code.

Stage 4 — Persistence (optional)

Adds durability so data survives a restart, and introduces trade-offs around write cost.

- Write-ahead log (WAL): append every **SET** / **DEL** to a log file before applying it in memory
- On startup, replay the log to rebuild state
- Explore durability trade-offs: fsync frequency, write batching, log compaction

Stage 5 — Extras

Pick whichever of these sound most interesting — none are required:

- TTL / key expiry, swept by a background thread
- Binary protocol instead of text, for faster parsing
- LRU eviction policy with a configurable max memory cap
- Basic pub/sub (**SUBSCRIBE** / **PUBLISH**) — forces you to think about broadcasting across open connections
- A benchmark tool to measure throughput (ops/sec) under concurrent load, then compare locking strategies against each other

Suggested Project Structure

```
kv-store/
|-- CMakeLists.txt
|-- src/
|   |-- main.cpp
|   |-- server.cpp / server.hpp      # socket handling, accept loop
|   |-- store.cpp / store.hpp        # hash map + locking strategy
|   |-- command_parser.cpp / .hpp
|   |-- wal.cpp / wal.hpp            # write-ahead log (stage 4)
|-- tests/
`-- benchmarks/
    |-- load_test.cpp
```

Milestone Checklist

Stage	Deliverable	Key concept
1	Working local CLI store	Data structures / parsing
2	Server reachable via nc/telnet	Sockets, networking
3	Concurrent clients, no data races	Thread pools, locking
4	Data survives a restart	Durability, WAL

5	One or more extras implemented	Your choice
---	--------------------------------	-------------

Notes

- Build with CMake so the project opens cleanly in CLion (or any other IDE) with no extra setup
- Get each stage working and committed before moving to the next — resist the urge to add concurrency before the single-threaded version works
- Stage 3 is the most valuable stage for someone who already understands thread pools; consider spending the most time there